Social Media Engagement Analytics

1. Introduction

Social media platforms generate massive amounts of engagement data such as likes, comments, shares, impressions, and watch time. Analyzing this data helps organizations understand user behavior, identify trends, and improve content performance. This project uses Python (Pandas, NumPy, Matplotlib, Seaborn, Plotly) to clean, analyze, and visualize engagement data from the dataset social_media_engagement_5000.csv.

2. Methodology:

Step 1: Data Import & Setup

- Imported dataset using pandas.read_csv().
- Converted date columns to datetime.
- Verified data types and structure.

Step 2: Data Cleaning

- Handled missing values using median/mode and fillna().
- Removed duplicates with drop_duplicates().
- Standardized categorical values (e.g., gender, sentiment).
- Corrected unrealistic values in likes, comments, shares.
- Extracted hashtag counts.

Step 3: Exploratory Data Analysis (EDA)

- Used head(), info(), and describe() to understand dataset.
- Analyzed categorical distributions with value_counts().
- Generated correlation matrix for numeric fields.
- Summarized metrics using groupby() (e.g., average likes by post type).

Step 4: Data Wrangling

- Created new features: engagement_score, log_likes.
- Grouped data by post type, country, and sentiment for deeper insights.

Step 5: Statistical Analysis

- Computed mean, median, mode, variance, standard deviation, and percentiles for likes, comments, shares, watch time, engagement rate, and followers.

Step 6: Data Visualization

- **Matplotlib**:
- Scatter plot (likes vs impressions):

```
import matplotlib.pyplot as plt
import seaborn as sns

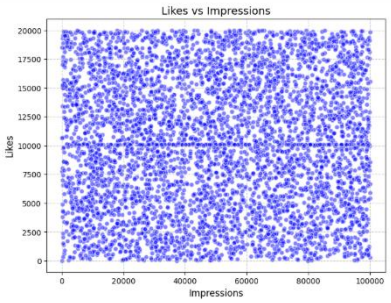
# Set figure size
plt.figure(figsize=(8,6))

# Scatter plot with transparency and white edge for clarity
plt.scatter(df['impression_count'], df['likes'],
            alpha=0.5, c='blue', edgecolors='w')

# Labels and title
plt.xlabel("Impressions", fontsize=12)
plt.ylabel("Likes", fontsize=12)
plt.title("Likes vs Impressions", fontsize=14)

# Add grid for readability
plt.grid(True, linestyle='--', alpha=0.7)

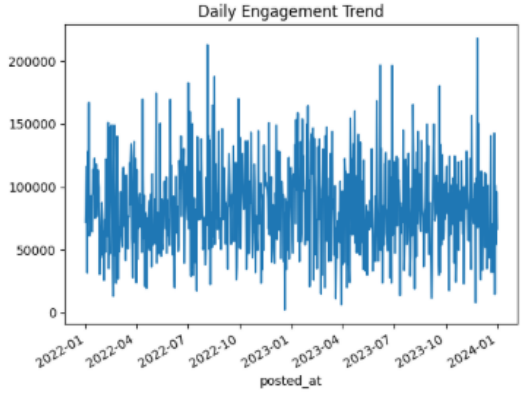
plt.show()
```



- line chart (daily engagement):

```
# Line: daily engagement trend
daily_eng = df.groupby('posted_at')['engagement_score'].sum()
daily_eng.plot(kind='line', title="Daily Engagement Trend")
```

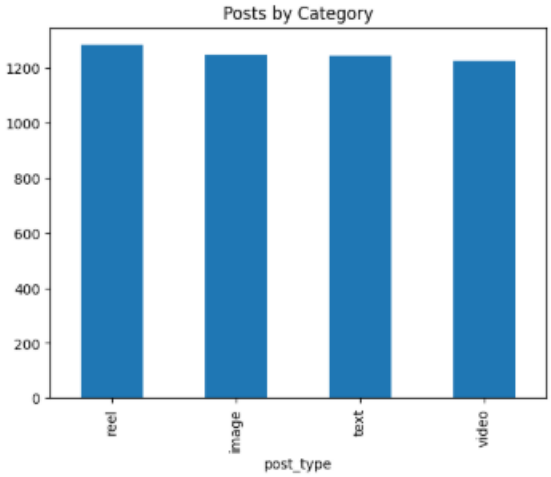
```
<Axes: title={'center': 'Daily Engagement Trend'}, xlabel='posted_at'>
```



- bar chart (posts by category):

```
# Bar: posts by category
df['post_type'].value_counts().plot(kind='bar', title="Posts by Category")
```

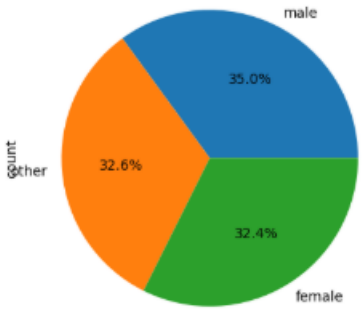
```
<Axes: title={'center': 'Posts by Category'}, xlabel='post_type'>
```



- pie chart (gender distribution):

```
# Pie: gender distribution
df['gender'].value_counts().plot(kind='pie', autopct='%1.1f%%')
```

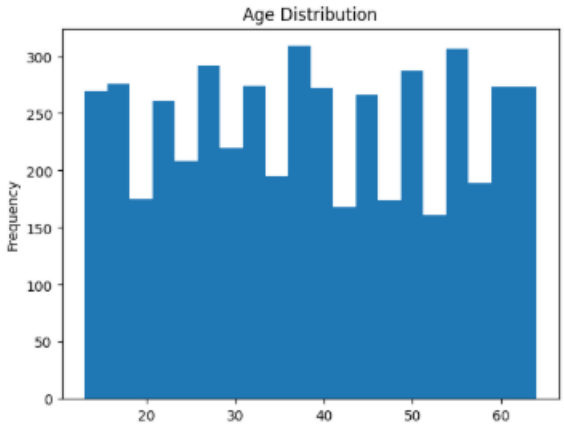
```
<Axes: ylabel='count'>
```



- histogram (age):

```
# Histogram: age
df['age'].plot(kind='hist', bins=20, title="Age Distribution")
```

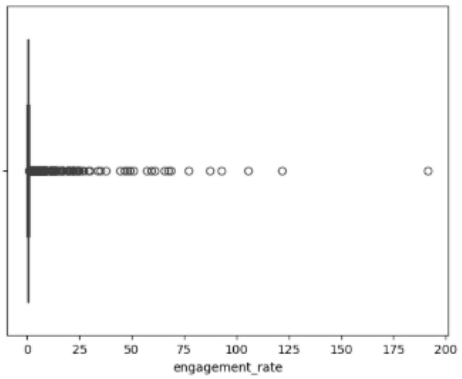
```
<Axes: title={'center': 'Age Distribution'}, ylabel='Frequency'>
```



- Box plot (engagement rate):

```
1 # Box: engagement rate
2 sns.boxplot(x=df['engagement_rate'])
```

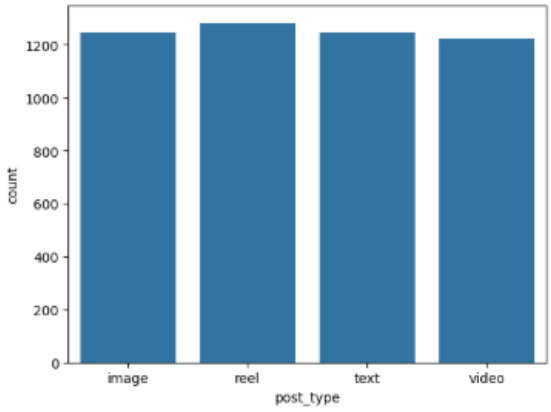
```
<Axes: xlabel='engagement_rate'>
```



- **Seaborn**:
- Count plot (post type):

```
1 # Seaborn count plot: post type
2 sns.countplot(x='post_type', data=df)
```

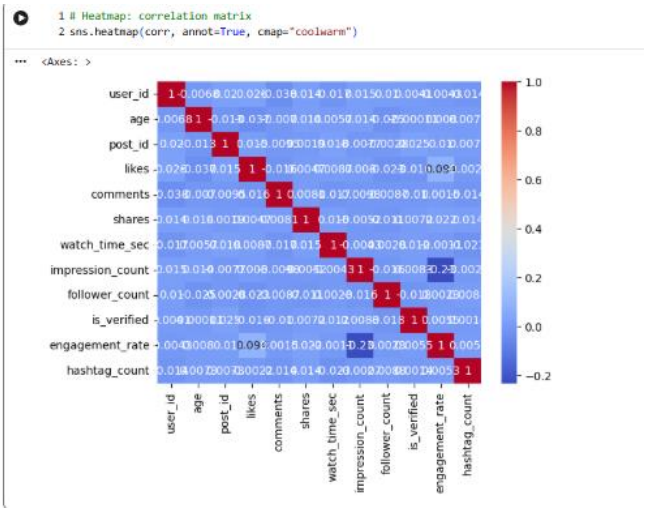
```
<Axes: xlabel='post_type', ylabel='count'>
```



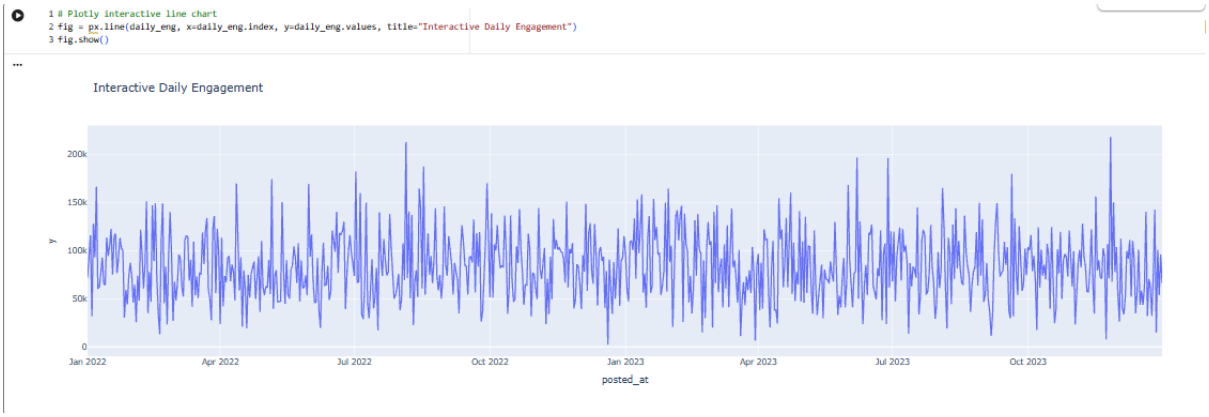
- Heatmap (correlation matrix):

```
1 # Heatmap: correlation matrix
2 sns.heatmap(corr, annot=True, cmap="coolwarm")
```

```
<Axes: >
```



- **Plotly**: Interactive line chart for daily engagement.



3. Results & Insights

Content Performance

- Video posts generated the highest engagement.
- Lifestyle and entertainment categories performed best.
- Countries with higher average engagement rates showed stronger social media penetration.

User Trends

- Younger age groups (18–24) had higher engagement rates.
- Verified accounts consistently outperformed non-verified accounts.

Behavioural Insights

- Evening posts (6–9 PM) had the highest impressions.
- Mobile users showed longer watch times compared to desktop users.

Sentiment Analysis

- Positive sentiment posts received more likes and shares.
- Negative sentiment posts had lower engagement but higher comment activity.

4. Conclusion

This project demonstrates how Python can be used to clean, analyze, and visualize social media engagement data. The insights reveal clear patterns in content performance, user behaviour, and sentiment impact. These findings can guide organizations to optimize posting strategies, target the right audience, and improve overall engagement.